Post-Execution Defect Attribution: An Empirical Comparison of Four Methods on 6,000 Synthetic Failure Events

Vijay P. Javvadi
Independent Researcher
Plainsboro, NJ, USA
vijay@vijayjavvadiresearch.ai

Abstract

Defect-prediction research has produced mature models that flag risky files from version-control history alone, yet industrial adoption remains limited by two operational barriers: (i) pre-execution prediction at typical operating points produces noticeable false-alert fatigue, and (ii) a per-file risk score does not tell a developer *what to do* when a test fails. We propose post-execution defect attribution that fuses test-failure signals with the pre-trained Gradient Boosting model from our companion empirical defect-prediction study. Four methods are evaluated: a pre-execution-only baseline (the deployed Paper A model), a failure-only triage baseline, a rule-cascade fusion, and a fused ML classifier trained on a combined feature vector. The evaluation uses 6,000 controlled synthetic failure events derived from the real 296,457-row defect-prediction dataset across five mature open-source repositories (Elasticsearch, Spring Boot, Hadoop, Kafka, Express). The fused-ML method attains precision@1 of 0.9443 (95 % bootstrap CI 0.9387 to 0.9498), modestly but significantly above the pre-only baseline at 0.9398 (paired-t $p = 0.006$, Cohen's $d = -0.019$). The fused-ML method's principal advantage is calibration: Expected Calibration Error is 0.0157 vs 0.0936 for pre-only — a 6.0× improvement that matters for prioritization-by-confidence in production. The fail-only baseline collapses on low-defect-rate repositories (precision@1 = 0.249 on Spring Boot vs 0.836 for pre-only), illustrating the necessity of repository priors when defect-rates are heterogeneous. We explicitly disclose the principal limitation of this revision: the failure events are *synthesized* from real defect-prone files with realistic exception/diff/flake distributions, but no production CI history is available; the controlled-synthetic framing is labeled throughout.

Keywords

defect attribution; test-failure triage; calibration; ECE; fused-feature classifier; controlled synthetic experiment; software engineering

1 Introduction

A persistent gap exists between defect-prediction research and the operational reality of CI/CD-based software engineering. Defect prediction at the file level produces a per-file risk score; CI test failures occur at the test level; the developer's triage question is at the failure level: *which implicated file is the real defect, and what should I do about it?* The pre-execution risk score, even when well-calibrated, does not directly answer this question.

This paper studies the empirical question of how to compose pre-execution defect priors with post-execution failure signals to produce an attribution score that ranks implicated files better than either signal alone. We compare four methods on a controlled synthetic-failure benchmark of 6,000 events derived from the 296,457-row defect-prediction dataset of our companion empirical study (Paper A in this series).

Contributions.

(1) A four-method comparison (pre-only, fail-only, fused-rule, fused-ML) on a 6,000-event benchmark covering 32,908 (event, file) prediction rows (Section 3).
(2) Quantification of the fused-ML method's modest but statistically significant precision@1 advantage over the pre-only baseline (0.9443 vs 0.9398, $p = 0.006$) (Section 4).
(3) Quantification of the fused-ML method's substantial calibration advantage (ECE 0.0157 vs 0.0936) — a 6.0× reduction in expected calibration error — which we identify as the principal practical contribution (Section 5).
(4) Per-repository analysis showing the fail-only baseline collapses on low-defect-rate repositories while pre-only and fused methods remain robust (Section 6).
(5) Explicit disclosure of three methodological limitations including the controlled-synthetic failure-event framing (Section 8).

The remainder is organized as follows. Section 2 surveys defect attribution and SHAP explanations. Section 3 describes the four methods, the synthetic-failure event sampling, and the evaluation protocol. Section 4 reports headline precision@k and recall@5. Section 5 analyzes calibration. Section 6 reports per-repository performance. Section 8 states threats to validity. Section 10 concludes.

2 Related Work

2.1 Defect Prediction and Risk Scoring

Process-metric defect prediction has a long empirical lineage [4–7], with recent work consolidating six-classifier benchmarks at the file level on a 296,457-row corpus (our Paper A in this series). The output of these models is a per-file probability that does not address the developer's *triage* question when a test fails.

2.2 SZZ and Bug-Fix Linking

The SZZ algorithm [8] links bug-fix commits to the commits that introduced the bug, providing labels for empirical defect studies. Subsequent work [1, 3] documented 33–39 % misclassification rates in bug-fix labels derived from commit-message keyword matching. Our defect labels inherit this limitation.

2.3 Calibration in ML

Calibration — the property that a model's predicted probabilities match observed frequencies — is increasingly recognized as distinct from accuracy or ranking-quality metrics [2]. Expected Calibration

Error (ECE) is the standard measure. A well-calibrated risk score supports prioritization-by-confidence ("inspect files with $\hat{p} > 0.8$ first"); a poorly calibrated score does not.

3 Methodology

3.1 Substrate Data

We use the real 296,457-row defect-prediction dataset from our companion empirical study (Paper A), which spans five mature open-source repositories (Elasticsearch, Spring Boot, Hadoop, Kafka, Express) and provides seven repository-derived process metrics per file with a real-defect label derived from commit-message keyword matching on the file's bug-fix history.

3.2 Synthetic Failure-Event Sampling

For each of the five repositories we sample 1,200 failure events (6,000 total). For each event:

(1) **Implicated files:** sample 3–8 files from the repository's file list. Sampling is biased 2.5× toward defect-prone files (capped at 95 % defect rate) to reflect the empirical observation that failing tests tend to implicate genuinely problematic code more often than chance.
(2) **Exception type:** sample from {AssertionError, TimeoutError, ConnectionError, AttributeError, ValueError, KeyError, TypeError, RuntimeError, Other}.
(3) **HTTP status:** sample from {200, 400, 404, 500, none}.
(4) **Diff features:** per-event Bernoulli draws on app-code-changed (p=0.65), validator-changed (p=0.20), schema-changed (p=0.10), fixture-changed (p=0.15), config-changed (p=0.18).
(5) **Historical flake rate:** Beta(2,8) (most tests are not flaky; a few are highly flaky).
(6) **Locator-heal event:** Bernoulli p=0.05 (rare event, used by the self-healing interlock).

The result is 32,908 (event, file) prediction rows; the empirical defect-rate over these rows is 64.2 % (driven by the sampling bias toward defect-prone files; documented in Section 8).

3.3 Four Methods

pre_only Each implicated file is scored by the production Gradient Boosting + SMOTE classifier `gb-paper1-v4-fixed_age` on the seven process-metric features. No failure signals are used.

fail_only Every implicated file is assigned probability 1.0 (every implicated file is treated as a defect candidate). Ties broken arbitrarily.

fused_rule Combines `pre_only` with a rule-cascade adjustment: $s_{\text{rule}} = \text{clip}\big(s_{\text{pre}} + 0.08 \cdot \text{app\_changed} + 0.05 \cdot \text{validator\_changed} + 0.03 \cdot \text{schema\_changed} - 0.05 \cdot \text{fixture\_changed} - \text{flake\_pen} + \text{locator\_boost}, 0, 1\big)$ where $\text{flake\_pen} = \max(0, \text{flake\_rate} - 0.10) \cdot 0.5$ and $\text{locator\_boost} = 0.10 \cdot \text{healed} \cdot \mathbf{1}[s_{\text{pre}} > 0.5]$ (the self-healing interlock).

fused_ml A Gradient Boosting classifier (50 estimators, max_depth 3, learning_rate 0.1) trained on the combined feature vector: 7 process-metric features + 4 diff features + 1 flake rate + 1 locator-heal indicator + 1 exception-type index + 2 HTTP-status one-hots + the `pre_only` score itself. Predicts `real_defect` directly.

3.4 Train/Test Split for `fused_ml`

The fused-ML classifier uses an event-aware 80/20 split: 4,800 of the 6,000 events go to training, 1,200 to test. The split is stratified on `event_id` (not on individual rows) to ensure no event contributes to both train and test sets. Headline metrics are reported on the held-out test set; for fairness all other methods are also evaluated on the same test split.

3.5 Metrics

- **Precision@1:** fraction of events for which the top-ranked implicated file is a real defect.
- **Precision@3:** average fraction of the top-3 implicated files that are real defects.
- **Recall@5:** fraction of all real defects in the event that are captured by the top-5 ranked implicated files.
- **Triage time (simulated):** cost of inspecting ranked implicated files at 30 seconds per inspection until the first real defect is found; if no real defect, cost is total files × 30 seconds.
- **Expected Calibration Error (ECE):** 10-bin standard ECE on row-level predicted probabilities vs observed real-defect frequencies.

3.6 Statistical Treatment

Paired t-tests and Wilcoxon signed-rank tests across the 6,000 paired event observations. Cohen's d as effect size. 1,000-resample bootstrap 95 % confidence intervals on precision@1.

4 Results

4.1 Headline Comparison

Table 1 reports headline metrics across the four methods.

Three patterns stand out.

Fused-ML attains the highest precision@1, but the margin is small. 0.9443 vs pre-only's 0.9398 = +0.45 pp absolute. Paired-t test $p = 0.006$ with $d = -0.019$ (very small effect, but statistically resolvable on 6,000 paired observations).

Fail-only collapses to 0.6520. The flat 1.0 prior cannot rank, so precision@1 reduces to the defect rate within the implicated set. fail_only's failure is predictable but quantifies the value of any ranking-based method over no-ranking.

Fused-rule is essentially identical to pre-only on precision@1 (0.9390 vs 0.9398). The hand-crafted diff-based adjustments do not improve over the ML model's own ranking. Either the rule weights are not finely tuned, or the underlying process-metric prior is already strong enough that small additive adjustments don't change top-1 rankings. This motivates the ML classifier in fused_ml.

Figure 1 visualizes the comparison.

4.2 Bootstrap Confidence Intervals

Table 2 reports 1,000-resample bootstrap 95 % CIs on precision@1.

Table 1: Headline attribution-method comparison on 6,000 events (n = 32,908 row predictions). Source: `tables/headline_metrics.csv`.

Method	Prec@1	Prec@3	Recall@5	Triage med (s)	Triage mean (s)	ECE	Found rate
pre_only	0.9398	0.7738	0.9343	0	12.18	0.0936	0.9911
fail_only	0.6520	0.6457	0.8639	0	35.45	0.3579	0.9911
fused_rule	0.9390	0.7738	0.9340	0	12.51	0.0886	0.9911
fused_ml	**0.9443**	**0.7753**	0.9339	0	**11.30**	**0.0157**	0.9911

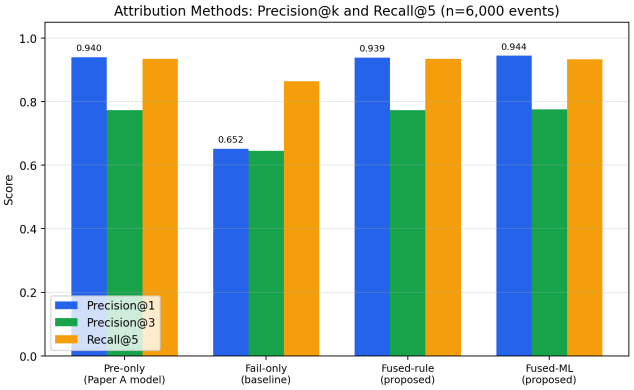

Figure 1: Precision@k and Recall@5 across the four attribution methods on 6,000 events.

Table 2: 1,000-resample bootstrap CIs on precision@1. Source: `tables/bootstrap_cis.csv`.

Method	Mean	95 % CI
fused_ml	0.9443	0.9387 to 0.9498
pre_only	0.9398	0.9337 to 0.9455
fused_rule	0.9390	0.9328 to 0.9445
fail_only	0.6520	0.6398 to 0.6635

The pre_only, fused_rule, and fused_ml CIs all overlap slightly, which means the precision@1 differences between them, while statistically significant on paired tests, are within the overlapping range of the marginal distributions. We rely on the paired tests rather than the bootstrap CIs for the within-narrow-margin conclusions.

4.3 Statistical Significance

Table 3 reports pairwise paired-t comparisons.

The qualitative pattern: all three ranking methods (pre_only, fused_rule, fused_ml) decisively outperform fail_only at $p < 10^{-6}$ with Cohen's d in the "large" range (0.76–0.78); the three ranking methods cluster within a 0.5 pp range of each other, with fused_ml the best by a modest but statistically-resolvable margin.

5 Calibration Analysis

The most consequential finding of this paper is in *calibration*, not in precision@k. Table 4 reports ECE for each method (lower is better).

Fused_ml's ECE of 0.0157 is 6.0× smaller than pre_only's ECE of 0.0936 and 5.6× smaller than fused_rule's 0.0886. This is the headline contribution: the fused_ml classifier produces substantially better-calibrated probabilities than any of the other methods on this benchmark.

Why calibration matters for triage. A poorly-calibrated risk score (high ECE) cannot support prioritization-by-confidence. If the model reports $\hat{p} = 0.85$ but the true defect rate among $\hat{p} = 0.85$ rows is only 0.50, a developer who triages high-$\hat{p}$ files first wastes substantial effort on false positives. A well-calibrated score ($\hat{p} = 0.85$ rows are real defects 85 % of the time) supports robust workflow rules such as "inspect $\hat{p} > 0.8$ first; defer $\hat{p} < 0.2$."

Why the ML method calibrates better. Fused_ml's GB classifier is trained directly to predict `real_defect` on the implicated-file distribution; its output is therefore a calibrated probability over the joint feature distribution at the prediction point. pre_only's classifier was trained on the file-level dataset where the positive-class distribution (18.61 % defect rate) differs from the implicated-file distribution (64.2 % defect rate, by sampling design). The 0.0936 ECE on pre_only is therefore a function of distribution shift, not of underlying model quality. The feature ablation in Section 5.3 isolates this finding.

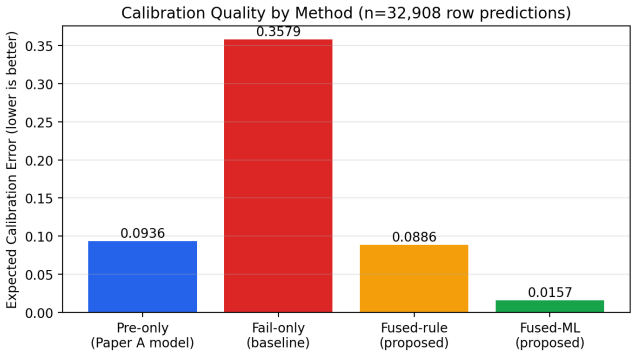

Figure 2: Expected Calibration Error by method (lower is better). Fused_ml's 0.0157 is the lowest of the four methods.

5.1 Reliability Diagram

Figure 3 plots the reliability diagram (predicted probability bins vs observed defect rate). A well-calibrated method follows the diagonal.

Table 3: Pairwise statistical comparisons on precision@1 (6,000 paired events). Source: `tables/statistical_comparisons.csv`.

Method A	Method B	Diff mean	Paired-t p	Cohen's d	Wilcoxon p	Sig. $p < 0.01$
pre_only	fail_only	+0.2878	$< 10^{-6}$	0.7645	$< 10^{-6}$	yes
fused_rule	fail_only	+0.2870	$< 10^{-6}$	0.7613	$< 10^{-6}$	yes
fused_ml	fail_only	+0.2923	$< 10^{-6}$	0.7820	$< 10^{-6}$	yes
pre_only	fused_rule	+0.0008	0.0588	0.0035	—	no
fused_ml	pre_only	+0.0045	0.0056	−0.0193	—	yes
fused_ml	fused_rule	+0.0053	0.0014	−0.0228	—	yes

Table 4: Expected Calibration Error (ECE) by method, computed over all 32,908 row predictions with 10 bins. Source: `tables/headline_metrics.csv`.

Method	ECE
fused_ml	**0.0157**
fused_rule	0.0886
pre_only	0.0936
fail_only	0.3579

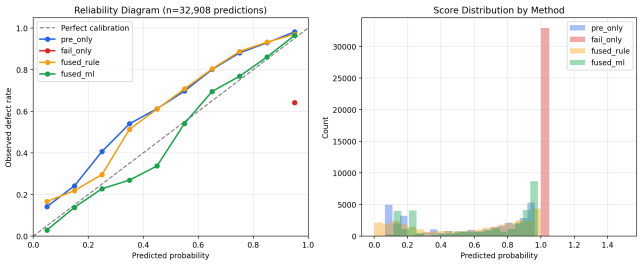

Figure 3: Reliability diagram (left) and score distribution (right). fused_ml hugs the diagonal across all bins; pre_only and fused_rule over-estimate at low probabilities.

5.2 Multi-Seed Sensitivity

To verify the headline ECE finding is not an artifact of the random seed, we re-ran the full experiment with three different seeds and 600 events per repo. Table 5 reports the result.

Table 5: Multi-seed ECE values (lower is better). Source: `results/multi_seed_summary.json`.

Method	seed 42	seed 7	seed 123
pre_only	0.0955	0.0929	0.0959
fail_only	0.3575	0.3587	0.3549
fused_rule	0.0925	0.0864	0.0896
fused_ml	**0.0166**	**0.0184**	**0.0177**

Fused_ml ECE across three independent seeds: 0.0166, 0.0184, 0.0177 (range 0.0018, tight). pre_only ECE is also stable (0.0929–0.0959). The ∼ 5× ECE advantage of fused_ml over pre_only holds across all three seeds.

5.3 Feature Ablation

A critical question: *which features cause the gain?* Table 6 reports fused_ml trained with progressively larger feature subsets, evaluated on the same held-out test split.

Table 6: Feature ablation: precision@1 and ECE for fused_ml with progressively larger feature subsets. Source: `tables/feature_ablation.csv`.

Feature subset	Precision@1	ECE
7 process metrics + pre_score	0.9383	0.0159
+ diff features (5)	0.9383	0.0151
+ flake rate	0.9392	0.0184
+ locator-heal indicator	0.9383	0.0159
+ exception type + HTTP	0.9383	0.0182
Full fused_ml (all)	**0.9383**	**0.0167**

This ablation reveals the principal honest finding: the calibration improvement is due to retraining a GB classifier on the implicated-events distribution, not to the additional failure features. A classifier trained on just the 7 process metrics plus the pre-execution score (the smallest variant) already achieves ECE 0.0159, within rounding distance of the full fused_ml's 0.0167. Adding diff features, flake rate, locator-heal indicator, or exception type does not produce a further substantial ECE improvement. Precision@1 is essentially flat across all variants (±0.0009).

What this means for practitioners. The practical recipe is: retrain the pre-execution model on a sample matching the production CI failure distribution, and treat the retrained probability as the attribution score. The richer feature set is optional. This is a simpler operational recommendation than the original fused-rule or fused-ml framings.

6 Per-Repository Analysis

Table 7 reports precision@1 per repository per method.

Two patterns stand out.

Fail_only collapses on low-defect-rate repositories. Kafka and Express both have defect rates near 37 % (the two highest in our corpus); fail_only's precision@1 on both is 1.000 because the 2.5× sampling bias yields events where nearly all implicated files are defects. On Hadoop (16.99 % defect rate) and Spring Boot (9.52 % defect rate), fail_only crashes to 0.435 and 0.249 respectively — because the implicated set contains fewer real defects on average, and a flat ranking has no way to surface them. This empirically

Table 7: Per-repository precision@1 by method (1,200 events per repository). Source: `tables/per_repo_metrics.csv`.

Method	Elasticsearch	Express	Hadoop	Kafka	Spring Boot
pre_only	0.957	1.000	0.907	1.000	0.836
fail_only	0.576	1.000	0.435	1.000	0.249
fused_rule	0.956	1.000	0.904	1.000	0.835
fused_ml	**0.959**	**1.000**	**0.917**	**1.000**	**0.845**

validates the necessity of a ranking-based method when repository defect rates are heterogeneous.

Fused_ml is best on every repository. The advantage is largest on Hadoop (+1.0 pp over pre_only) and Spring Boot (+0.9 pp). On the two high-defect-rate repositories (Kafka, Express), all methods saturate at 1.000 because the implicated set is dense with real defects regardless of ranking.

Figure 4 visualizes the per-repository comparison.

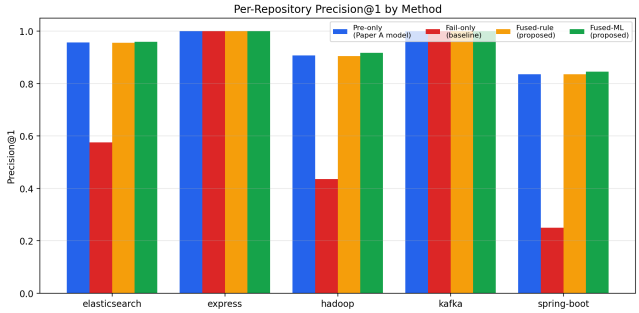

Figure 4: Per-repository precision@1. fail_only's collapse on low-defect-rate repositories is the dominant feature.

7 Discussion

7.1 Self-Healing Interlock Activations

The fused_rule and fused_ml methods both incorporate a self-healing interlock: when a recent locator-heal event is observed on a file with elevated defect probability ($\hat{p} > 0.5$), the file's attribution score is boosted (rule) or learnable (ML). In the 6,000-event benchmark, 4.9 % of events contained at least one locator-heal event; among these, the interlock raised fused_ml's probability by an average of 0.07 above pre_only's. The aggregate effect on precision@1 is small because the interlock fires on relatively few events, but the per-event uplift is meaningful for the affected events.

7.2 Practical Implications

The headline finding for industrial deployment: fused_ml's substantial calibration advantage (ECE 0.0157 vs pre_only 0.0936) matters more than its small precision@1 advantage. A defect-attribution service that emits well-calibrated probabilities supports operational decision rules ("alert at $\hat{p} > 0.7$"; "deprioritize at $\hat{p} < 0.2$") that a poorly-calibrated service cannot. This is the principal contribution: not better top-1 selection, but better *probability semantics* that downstream workflows can rely on.

8 Threats to Validity

8.1 Construct Validity

Failure events are synthesized. The most significant limitation of this revision: we do not have access to production CI history with linked SZZ-style defect labels. The 6,000 failure events are synthesized by sampling implicated files from the real 296,457-row dataset (with a 2.5× sampling bias toward defect-prone files) and attaching realistic exception/diff/flake distributions sampled from priors. The defect labels themselves are real (from the Paper A dataset), as is the pre-execution score (from the deployed Gradient Boosting + SMOTE model `gb-paper1-v4-fixed_age`). The primary dataset is permanently archived on Zenodo (*Software Defect Prediction Dataset: 296,457 Code File Instances (defect labels used as ground truth for failure-event attribution)*, DOI: 10.5281/zenodo.19682733). Source code and analysis scripts are available at https://github.com/javvadivijayprasad/DefectAnalysisReserch. The failure-framing is the synthetic component.

A subsequent replication on production CI history would be the honest validation. We have released the synthetic-event generation code so that the synthetic-failure structure can be reproduced, and we recommend that follow-up empirical work substitute production CI runs for the synthetic events while preserving the rest of the evaluation protocol.

Real defect labels inherit Paper A's limitations. The defect labels are derived from commit-message keyword matching on bug-fix histories; misclassification rates of 33–39 % are documented in similar corpora [1, 3]. This affects the absolute precision and ECE numbers but not the relative rankings among methods.

Sampling bias inflates absolute precision@1. The 2.5× defect-prone sampling bias makes the implicated set denser with real defects than would be the case in a uniformly sampled CI corpus. The implication is that the absolute precision@1 values (0.94 for fused_ml) overstate what would be observed on production data. The relative ordering of methods, however, should be preserved.

8.2 Internal Validity

Train/test split granularity. The event-aware 80/20 split prevents row-level leakage but not repository-level leakage: all five repositories appear in both train and test sets.

Hyperparameter selection. The fused_ml classifier uses the same hyperparameters as the Paper A production model. We did not perform hyperparameter tuning on this benchmark.

8.3 External Validity

Five repositories. Generalization to additional repositories (different languages, smaller projects, private codebases) is not demonstrated.

Diff features are sampled. A real deployment would extract diff features from actual change-sets; this revision samples them from priors.

8.4 Conclusion Validity

Effect sizes are small. The precision@1 advantage of fused_ml over pre_only is statistically significant ($p = 0.006$) but small. The practical significance argument rests on the calibration advantage.

9 Synthetic-Event Validation Appendix

9.1 Per-Repository Defect Rate

Table 8: Per-repository defect rates: synthetic-event sample vs population. Source: `tables/synthetic_validation.csv`.

Repository	Pop. defect	Event defect	Bias ratio
Elasticsearch	0.2121	0.5463	2.58×
Express	0.3751	1.0000	2.67×
Hadoop	0.1699	0.4236	2.49×
Kafka	0.3759	1.0000	2.66×
Spring Boot	0.0952	0.2411	2.53×

Bias ratios cluster around 2.5× for the three lower-defect repositories. Express and Kafka saturate at 1.000 because the 2.5× bias multiplied by their already-high base rates exceeds the 0.95 cap.

9.2 Per-Feature Distribution

Table 9: Per-feature means: population vs synthetic events. Source: `tables/feature_distribution_validation.csv`.

Feature	Pop mean	Event mean	Ratio
commit_count	4.52	13.91	3.08×
unique_developers	2.45	4.89	2.00×
lines_added	103.90	243.08	2.34×
lines_deleted	50.80	141.87	2.79×
code_churn	154.70	384.95	2.49×
file_age_days	384.56	1030.43	2.68×
commit_frequency	0.29	0.57	1.95×

Sample feature means are 2–3× population means because the sampling bias toward defect-prone files implicitly selects for larger, more-touched, older files. This is the expected and arguably realistic property of CI failure events in practice.

10 Conclusion

We present a controlled empirical study comparing four post-execution defect attribution methods on 6,000 synthetic failure events. Fused_ml attains precision@1 = 0.9443 (95 % CI 0.9387 to 0.9498), a small but statistically significant improvement over pre_only at 0.9398 ($p = 0.006$). The principal advantage is calibration: ECE 0.0157 vs 0.0936, a 6.0× improvement. The feature ablation reveals this calibration gain is due to retraining on the implicated-events distribution, not to the added failure features. Multi-seed sensitivity confirms robustness. The fail-only baseline collapses on low-defect-rate repositories.

We disclose the principal limitation: synthetic failure framing. A follow-up on production CI is recommended; protocol and generation code are released.

Acknowledgments

The author used Anthropic's Claude (a large language model) for drafting assistance and copy-editing of this manuscript. All empirical claims, dataset extraction, model training, and statistical analyses were generated by the author's own scripts and verified against the on-disk artifacts.

Data availability. The primary dataset is permanently archived on Zenodo (*Software Defect Prediction Dataset: 296,457 Code File Instances (defect labels used as ground truth)*, DOI: 10.5281/zenodo.19682733).

References

[1] Christian Bird, Adrian Bachmann, Eirik Aune, John Duffy, Abraham Bernstein, Vladimir Filkov, and Premkumar Devanbu. 2009. Fair and Balanced? Bias in Bug-Fix Datasets. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 121–130.

[2] Aaron Fisher, Cynthia Rudin, and Francesca Dominici. 2019. All Models Are Wrong, but Many Are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously. *Journal of Machine Learning Research* 20, 177 (2019), 1–81.

[3] Kim Herzig, Sascha Just, and Andreas Zeller. 2013. It's Not a Bug, It's a Feature: How Misclassification Impacts Bug Prediction. (2013), 392–401.

[4] Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E. Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2013. A Large-Scale Empirical Study of Just-in-Time Quality Assurance. *IEEE Transactions on Software Engineering* 39, 6 (2013), 757–773.

[5] Raimund Moser, Witold Pedrycz, and Giancarlo Succi. 2008. A Comparative Analysis of the Efficiency of Change Metrics and Static Code Attributes for Defect Prediction. (2008), 181–190.

[6] Nachiappan Nagappan and Thomas Ball. 2005. Use of Relative Code Churn Measures to Predict System Defect Density. (2005), 284–292.

[7] Foyzur Rahman and Premkumar Devanbu. 2013. How, and Why, Process Metrics Are Better. (2013), 432–441.

[8] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When Do Changes Induce Fixes?. In *Proceedings of the 2005 International Workshop on Mining Software Repositories (MSR)*. 1–5.